Lab Exercises

Digital ASIC Design: RTL to GDS -II

General Instructions

This document contains 20 tasks divided into 6 exercises. For each task, you will modify files, run the flow, observe the results, and record your observation in your files.

How to run the flow:

- **Ubuntu:** `./flow.tcl -design alu`
- **Mac:** `librelane config.json`

How to clean previous runs (do this before starting a new task):

```
rm -rf runs/*
```

How to view the GDS layout and remove filler cells in KLayout: At the end of every task, you must view the final manufacturing file (.gds) in KLayout to see the physical gates. **Hide Filler Cells:** Open the **Cell Tree** panel on the left side of KLayout. Type `fill` in the search box. Right-click the matching `sky130_fd_sc_hd_fill_*` cells and select **Hide**. Do the same for `decap` and `tap` cells. This removes dummy elements so you can clearly see your actual logic gates.

—

1 Experiment 1: Floorplanning and Area Utilization

Objective: Understand how chip size and core packing affect the physical layout.

Task 1: Initial Compilation

Run the default 4-bit ALU flow using the `config.json`. Open `metrics.csv` and record `design_die_area`, `power_total`, and `magic_drc_error_count`. View the GDS in KLayout and hide the filler cells.

Task 2: High Core Utilization

Change `"FP_CORE_UTIL": 5` to `"FP_CORE_UTIL": 50` in `config.json`. Run the flow. Check if `magic_drc_error_count` increases due to routing congestion. View the GDS. Notice how tightly packed the standard cells are.

Task 3: Low Core Utilization

Change `"FP_CORE_UTIL": 50` to `"FP_CORE_UTIL": 2`. Run the flow. View the GDS. Notice the massive gaps between standard cells. Hide the filler cells to see how far apart the actual logic gates are placed.

Task 4: Absolute Die Area Sizing

Remove the "FP_CORE_UTIL" line entirely. Add:

```
    "FP_SIZING": "absolute",
    "DIE_AREA": "0 0 100 100",
```

Run the flow. You have forced the chip to be exactly 100x100 micrometers. Verify this dimension in KLayout.

—

2 Experiment 2: Power Distribution Network (PDN)

Objective: Observe the effects of power grid density.

Task 5: Sparse Power Grid

Change "FP_PDN_VPITCH" and "FP_PDN_HPITCH" to 150. Run the flow. Open the GDS. You will physically see the thick power straps are very far apart.

Task 6: Dense Power Grid

Change the pitches to 15. Run the flow. Open the GDS. Notice the high density of metal straps. This provides excellent voltage delivery but leaves less room for signal routing.

Task 7: PDN Offsets

Add "FP_PDN_VOFFSET": 10 and "FP_PDN_HOFFSET": 10 to config.json. Run the flow. Observe in KLayout how the starting position of the first power strap shifts from the origin.

—

3 Experiment 3: Clocking and Sequential Logic

Objective: Convert combinational logic to sequential and analyze timing constraints.

Task 8: Clock Bugs

Keep the Verilog combinational, but change "CLOCK_PORT": null to "CLOCK_PORT": "clk" in config.json. Run the flow. Observe the Clock Tree Synthesis (CTS) failure or warning in the terminal.

Task 9: Adding a Flip-Flop

Open alu.v and make it sequential:

```
module alu(
  input wire clk,
  input wire [3:0] a, b,
  input wire [2:0] opcode,
  output reg [3:0] result
);
  always @(posedge clk) begin
```

Keep "CLOCK_PORT": "clk" and add "CLOCK_PERIOD": 10.0. Run the flow. Open the GDS, hide filler cells, and locate the newly added flip-flops.

Task 10: Pushing Clock Frequency

Change `"CLOCK_PERIOD": 10.0` to `"CLOCK_PERIOD": 0.5`. Run the flow. Check `timing_setup_ws` in the CSV. A negative slack means the chip fails at 2 GHz.

Task 11: Clock Buffering

With the 0.5 period, observe the clock tree in the GDS layout. Notice how many clock buffer cells (search `clkbuf` in Cell Tree) the tool inserted to manage the fast clock.

—

4 Experiment 4: Synthesis Strategies

Objective: Control how the logic synthesizer chooses standard cells.

Task 12: Area Optimization

Add `"SYNTH_STRATEGY": "AREA 0"` to `config.json`. Run the flow. Record the total cell count and area from the CSV. View the GDS and observe the smaller gate sizes.

Task 13: Delay Optimization

Change to `"SYNTH_STRATEGY": "DELAY 0"`. Run the flow. Compare area with Task 12. The tool now uses larger, faster standard cells. View the GDS to see the increased gate dimensions.

Task 14: Max Fanout Limit

Add `"SYNTH_MAX_FANOUT": 4`. Run the flow. The tool will insert many extra buffer cells to ensure no single gate drives more than 4 other gates. Verify the increased buffer count in the GDS.

—

5 Experiment 5: Architecture and Data Path Scaling

Objective: Observe the physical cost of complex arithmetic operations.

Task 15: Adding a Multiplier

Change opcode `3'b111` in `alu.v` to `result = a * b;`. Run the flow. Check `design_die_area` and `power_total`. View the GDS to see the massive increase in logic gates required for multiplication.

Task 16: Scaling to 16-bit ALU

Change all `[3:0]` to `[15:0]`. Run the flow. Notice how routing congestion increases. Open the GDS and hide fillers.

Task 17: Scaling to 32-bit ALU

Change all `[15:0]` to `[31:0]`. Change `FP_CORE_UTIL` to 40 to give it room. Run the flow. The compilation will take much longer. View the sprawling layout in the GDS.

—

6 Experiment 6: Routing and Metal Layers

Objective: Understand interconnect constraints.

Task 18: Manhattan Routing Directions

Open KLayout. Hide all layers except `met1`. Note the direction of the wires (horizontal). Hide `met1` and show `met2` (vertical). This is the Manhattan routing grid.

Task 19: Restricting Max Routing Layer

Add `"RT_MAX_LAYER": "met3"` to `config.json`. Run the flow. Check `magic_drc_error_count`. By forcing wires into only 3 layers, DRC violations often occur.

Task 20: Minimum Routing Layer

Add `"RT_MIN_LAYER": "met2"`. Run the flow. The router will avoid using `met1` for global routing, leaving it purely for standard cell internal connections. View the GDS metal layers to verify.

—

A Source Codes and Basic Configurations

A.1 Combinational ALU Design (`alu.v`)

Use this as the starting point for Tasks 1-8.

```
`timescale 1ns/1ps

module alu(
  input wire [3:0] a, b,
  input wire [2:0] opcode,
  output reg [3:0] result
);

  always @(*) begin
    case (opcode)
      3'b000: result = a + b;
      3'b001: result = a - b;
      3'b010: result = a & b;
      3'b011: result = a | b;
      3'b100: result = a ^ b;
      3'b101: result = ~a;
      3'b110: result = a<<1;
      3'b111: result = a>>1;
      default: result = 4'd0;
    endcase
  end
endmodule
```

A.2 Sequential ALU Design (`alu.v`)

Use this for Tasks 9-11 when introducing flip-flops and clock signals.

```
`timescale 1ns/1ps

module alu(
  input wire clk,
  input wire [3:0] a, b,
  input wire [2:0] opcode,
  output reg [3:0] result
);

  always @(posedge clk) begin
    case (opcode)
      3'b000: result <= a + b;
      3'b001: result <= a - b;
      3'b010: result <= a & b;
      3'b011: result <= a | b;
      3'b100: result <= a ^ b;
      3'b101: result <= ~a;
      3'b110: result <= a<<1;
      3'b111: result <= a>>1;
      default: result <= 4'd0;
    endcase
  end
endmodule
```

A.3 Basic Configuration (`config.json`)

This is the foundational configuration file modified throughout the experiments.

```
{
    "DESIGN_NAME": "alu",
    "VERILOG_FILES": ["dir::src/alu.v"],
    "CLOCK_PORT": null,
    "FP_CORE_UTIL": 5,
    "FP_PDN_VPITCH": 25,
    "FP_PDN_HPITCH": 25
}
```

B Configuration Parameters Dictionary

The following list explains the function of each `config.json` variable used during these experiments. By tweaking these variables, you dictate how the EDA tools physically assemble your chip.

- **`DESIGN_NAME`**: Specifies the top-level module name matching your Verilog code. Used to name all output reports and databases.
- **`VERILOG_FILES`**: Points to the source RTL files to be synthesized. `dir::` acts as a relative path to the configuration file directory.
- **`CLOCK_PORT`**: Identifies which pin acts as the clock. Set to `null` for purely combinational designs so the tool skips clock tree synthesis.
- **`CLOCK_PERIOD`**: The target clock cycle time in nanoseconds (ns). E.g., `10.0` requests a 100 MHz clock. The tool attempts to optimize logic paths to meet this deadline.
- **`FP_CORE_UTIL`**: Floorplan Core Utilization (%). Dictates the density of the standard cells. Low values (e.g., 5) spread gates far apart; high values (e.g., 80) pack them tightly.
- **`FP_SIZING`**: Determines how the chip boundary is calculated. Defaults to relative (based on utilization), but setting to `"absolute"` allows you to manually draw the die boundary.
- **`DIE_AREA`**: Used with absolute sizing. Defines the coordinates of the chip's outer bounding box in micrometers `[x0, y0, x1, y1]`.
- **`FP_PDN_VPITCH / HPITCH`**: Sets the vertical and horizontal distance (pitch) between the VDD and GND metal straps in the Power Distribution Network.
- **`FP_PDN_VOFFSET / HOFFSET`**: Sets the starting offset from the edge of the die for the first vertical or horizontal power strap.
- **`SYNTH_STRATEGY`**: Controls Yosys synthesis behavior. `"AREA 0"` tells the synthesizer to prioritize selecting small standard cells to save space. `"DELAY 0"` prioritizes fast, high-drive-strength cells to maximize speed.
- **`SYNTH_MAX_FANOUT`**: Caps the maximum number of inputs a single logic gate is allowed to drive, forcing the insertion of buffer trees to share the electrical load.
- **`RT_MAX_LAYER`**: Restricts global routing to layers below or equal to the specified metal layer (e.g., `"met3"`), simulating chips with limited metallization stacks.
- **`RT_MIN_LAYER`**: Forces global routing to start on a higher metal layer, reserving lower layers strictly for standard cell internal geometry.